

CWG3103 “Corresponding members and by-value object parameters”

Document #: P4138R1 [[Latest](#)] [[Status](#)]
Date: 2026-03-24
Project: Programming Language C++
Audience: Core Working Group
Reply-to: Vlad Serebrennikov
<serebrennikov.vladislav@gmail.com>

Contents

- [1 Abstract](#)
- [2 Revision History](#)
- [3 Background](#)
- [4 Status quo](#)
- [5 Not all !\[\]\(38441ceaa711016e0bf2ad46ad394ff4_img.jpg\) are created equal](#)
- [6 Proposed changes](#)
- [7 Proposed wording](#)
 - [7.1 Option 1 \(preferred\)](#)
 - [7.2 Option 2](#)
 - [7.3 Annex C entry](#)
- [8 References](#)

§ 1 Abstract

While looking at [\[CWG3103\]](#), I got interested how we arrived at status quo, specifically how [\[basic.scope.scope\]/3.1](#) came to be, and this is what I found. One of the conclusions I arrived at is that it doesn’t make much sense to overload member functions with explicit object parameter of non-reference type with member functions of any other kind of object parameter with the same type, ignoring references.

§ 2 Revision History

R1:

- Minor changes to the wording after CWG review on 2026-03-24.

R0:

- Initial revision.

§ 3 Background

The intent to give member functions with no ref-qualifier special treatment can be tracked all the way to [\[N1821\] \(Extending Move Semantics To *this \(Revision 2\)\)](#), which introduced ref-qualifiers:

The `&` and `&&` suffixes are allowed only on members that allow a cv-qualifier in the same location (e.g., not constructors!). Additionally, neither of these forms can be overloaded with the existing form (the latter retains its binding semantics, including the exception that allows rvalues to be bound).

I think this was caused by the fact that implicit object parameter of a member function with no qualifiers can bind even rvalues, despite being of non-const lvalue reference type ([\[over.match.funcs\]/5](#)), which, I think, means the identity conversion and leads to ambiguous calls even without this restriction.

This, and the obvious rule that object parameters of the same type correspond, shape the status quo of the wording.

§ 4 Status quo

The table below summarizes whether object parameters of two declarations of member functions correspond (✗) or don't correspond (✓). Relevant wording is [\[basic.scope.scope\]/3](#) and [\[over.match.funcs\]/4](#).

	no ref	<code>&</code>	<code>&&</code>	<code>this D</code>	<code>this D&</code>	<code>this D&&</code>
no ref	✗					
<code>&</code>	✗	✗				
<code>&&</code>	✗	✓	✗			
<code>this D</code>	✗	✓	✓	✗		
<code>this D&</code>	✗	✗	✓	✓	✗	
<code>this D&&</code>	✗	✓	✗	✓	✓	✗

Implementations agree on 18 out of 21 cases ([Compiler Explorer](#)):

- GCC and MSVC fully conform;
- EDG has a bug where explicitness of the object parameter by itself is enough to break correspondence despite types being the same;

- Clang considers `this D` and `this D&` to correspond (but this might be a desirable direction as discussed in [Proposed changes](#) section).

However, when one of the declarations is named by a using-declaration, implementations agree only on 18 out of 36 cases, which includes 15 additional symmetrical cases left blank in the table ([Compiler Explorer](#)):

- MSVC struggles a lot;
- EDG struggles a lot, too, but less than MSVC;
- Clang conforms if using-declaration and declaration of member function are in the “right” lexical order, which is a unique behavior across surveyed implementations;
- GCC fully conforms.

Note that in this latter case correspondence (✗) doesn’t translate to program being ill-formed per [\[namespace.udecl\]/11](#).

§ 5 Not all are created equal

The table below summarizes whether a call to an overload set is well-formed depending on value category of the object argument, focusing on `this D`.

	By itself	With <code>this D</code>
no ref	Both	Conflicting
<code>&</code>	Lvalues	Rvalues
<code>&&</code>	Rvalues	Lvalues
<code>this D</code>	Both	Conflicting
<code>this D&</code>	Lvalues	Rvalues
<code>this D&&</code>	Rvalues	Lvalues

Note how adding a `this D` overload to an existing member function F flips the set of well-formed calls to its complement, because `this D` is best viable function in all cases, rendering F obsolete.

All implementations agree, with an exception of Clang, which consider `this D` and `this D&` to be corresponding object parameters.

§ 6 Proposed changes

Explicit object parameters of non-reference types should correspond with any other object parameters that have the same type ignoring references, i.e. it should be ill-formed to overload member functions with such object parameter. This will render the second example in [\[CWG3103\]](#) ill-formed as requested.

Worth mentioning that Clang already [rejects this D](#) and [this D&](#) overloads. Proposed change will make it conformant.

Table below reflects correspondence of object parameters with the proposed changes applied. Combinations that become correspondent are marked as **XX**.

	no ref	&	&&	this D	this D&	this D&&
no ref	XX					
&	XX	XX				
&&	XX	XX	XX			
this D	XX	XX	XX	XX		
this D&	XX	XX	XX	XX	XX	
this D&&	XX	XX	XX	XX	XX	XX

§ 7 Proposed wording

§ 7.1 Option 1 (preferred)

Change 6.4.1 [\[basic.scope.scope\]](#) paragraph 3 as follows:

- 3 Two non-static member functions have *corresponding object parameters* if
- (3.1) — ~~exactly one is an implicit object member function with no ref-qualifier and the types of their object parameters ([~~dcl.fct~~]), after removing references, are the same, or~~
 - (3.1) — the types of their object parameters ([[dcl.fct](#)], [[over.match.funcs](#)]) are equivalent~~., or~~
 - (3.2) — [if one of the functions is](#)
 - (3.2.1) — [an implicit object member function with no ref-qualifier or](#)
 - (3.2.2) — [a member function with an explicit object parameter whose type is not a reference type](#)
- [the types of their object parameters, after removing references, are equivalent.](#)

§ 7.2 Option 2

Change 6.4.1 [\[basic.scope.scope\]](#) paragraph 3 as follows:

- 3 Two non-static member functions have *corresponding object parameters* if
- (3.1) — exactly one is an implicit object member function with no ref-qualifier and the types of their object parameters ([[dcl.fct](#)]), after removing references, are the same, or

- (3.2) — one is a member function with an explicit object parameter whose type is not a reference type, and the types of their object parameters, after removing references, are equivalent, or
- (3.3) — the types of their object parameters are equivalent.

§ 7.3 Annex C entry

Add a new subclause to C.1 [diff.cpp23] for 6 [basic] with the following paragraph:

Affected subclause: [basic.scope.scope]

Change: Explicit object parameter of non-reference type corresponds with any other object parameter of the same type, ignoring references.

Rationale: If an overload set of member functions that differ only in their object parameter has a function with an explicit object parameter of non-reference type, other functions in the overload set cannot be selected by overload resolution.

Effect on original feature: A valid C++ 2023 program that declares such an overload set is ill-formed.

[*Example:*

```
struct S {  
    void f(this S);  
    void f() &;      // ill-formed; previously well-formed  
    void g(this S);  
    void g() &&;     // ill-formed; previously well-formed  
    void h(this S);  
    void h(this S&); // ill-formed; previously well-formed  
    void i(this S);  
    void i(this S&&); // ill-formed; previously well-formed  
};
```

— *end example*]

§ 8 References

[CWG3103] Corentin Jabot. 2025-11-02. Corresponding members and by-value object parameters.

<https://wg21.link/cwg3103>

[N1821] D. Vandevoorde, B. Kozicki. 2005-08-24. Extending Move Semantics To *this (Revision 2).

<https://wg21.link/n1821>